

Sí. Y de hecho, ahora que tenés **Claude Pro**, creo que conviene hacer algo distinto a lo que hicimos la primera vez: **antes de tocar código, definir bien el modelo de la aplicación y separar arquitectura, datos, lógica y diseño**. Así después Cursor puede encargarse de implementar sin que cada cambio termine rompiendo otra cosa.

Te dejo abajo un **informe base del proyecto**, pensado además para que puedas copiarlo casi directamente en Claude y usarlo como contexto inicial.

Proyecto: Sistema de Gestión para Caruso Propiedades

1. Objetivo general

Desarrollar una aplicación web interna para **Caruso Propiedades**, inmobiliaria ubicada en Paso del Rey, Moreno, que permita centralizar la administración diaria de propiedades, propietarios, inquilinos, contratos, cobranzas, deudores, servicios y liquidaciones.

La aplicación debe reemplazar progresivamente el uso de múltiples planillas, documentos y procesos manuales.

El objetivo **no es crear un software inmobiliario gigantesco y complejo**, sino una herramienta interna:

- rápida;
- clara;
- práctica;
- pensada principalmente para PC;
- fácil de aprender;
- con búsquedas rápidas;
- que reduzca carga administrativa;
- y que permita imprimir documentación cuando sea necesario.

La prioridad es **funcionalidad + estabilidad + facilidad de uso**, antes que sofisticación técnica.

2. Contexto de la inmobiliaria

Caruso Propiedades administra una cantidad importante de alquileres, del orden de **170 contratos/alquileres activos**, además de propiedades en venta y otros servicios.

Los alquileres no siguen todos el mismo esquema.

Existen contratos con:

- actualización por ICL;
- actualización por IPC;
- actualización cada 3 meses;
- actualización cada 4 meses;
- actualización cada 6 meses;
- otros esquemas particulares.

Por eso el sistema **no debe asumir que todos los contratos funcionan igual**.

La aplicación tiene que permitir configurar el mecanismo de actualización de cada contrato.

3. Filosofía de la aplicación

Una de las cosas más importantes aprendidas del primer intento es:

No romper funcionalidades existentes para conseguir una arquitectura "más linda".

La aplicación anterior empezó como un `index.html` muy grande, de aproximadamente 10.000 líneas.

Posteriormente se intentó ordenar/refactorizar el proyecto pasando parte de la lógica a Python y separando módulos.

El resultado fue que algunas funcionalidades y aspectos visuales que ya funcionaban comenzaron a romperse.

Por eso la nueva arquitectura debería partir de una filosofía:

Separación clara

```
Interfaz
  ↓
Lógica de negocio
  ↓
Base de datos
```

Y no:

```
HTML
├─ HTML
├─ JavaScript
```

- └─ datos
- └─ lógica
- └─ estilos
- └─ todo mezclado

La aplicación debería ser modular desde el principio.

4. Módulos principales

La aplicación debería tener como mínimo los siguientes módulos:

Dashboard

Pantalla inicial.

Debe permitir ver rápidamente el estado general de la inmobiliaria.

Ejemplos de información:

- contratos activos;
- alquileres;
- cobranzas pendientes;
- deudores;
- contratos próximos a vencer;
- actualizaciones de alquiler pendientes;
- tareas o alertas;
- propiedades disponibles;
- movimientos recientes.

No necesariamente todo tiene que estar implementado en la primera versión.

El dashboard debe funcionar como **centro de control**, no como una pantalla llena de estadísticas inútiles.

5. Propiedades

Este es uno de los módulos centrales.

Cada propiedad debería tener información como:

Identificación

- ID interno
- tipo de propiedad

- dirección
- localidad
- barrio
- código postal
- piso
- departamento
- unidad

Características

- dormitorios
- ambientes
- baños
- cochera
- patio
- terraza
- balcón
- metros cuadrados
- observaciones

Estado

Por ejemplo:

- Disponible
- Alquilada
- En venta
- Reservada
- En mantenimiento
- Baja

Pero el sistema debería permitir ampliar estos estados.

6. Búsqueda de propiedades

Esta era una de las funcionalidades que particularmente te gustaba y que **no había que eliminar durante los refactors**.

La idea es tener una búsqueda rápida tipo Google.

Por ejemplo:

Buscar propiedad...

Y que al escribir:

"Paso del Rey Belgrano"

encuentre propiedades relacionadas.

O:

"Moreno 1234"

O:

"duplex gaona"

O:

"Juan XXIII 450"

La búsqueda debería poder buscar sobre varios campos simultáneamente.

Idealmente:

Dirección
+
Localidad
+
Tipo
+
Propietario
+
Inquilino
+
ID

Y mostrar resultados instantáneamente.

7. Ficha de propiedad

Al seleccionar una propiedad debería abrirse una ficha completa.

Conceptualmente:

PROPIEDAD #0345
Duplex 2 ambientes
Paso del Rey

Estado: ALQUILADA
Datos de la propiedad
Dirección
Localidad
Características
Propietario
Nombre
DNI
Teléfono
Contrato actual
Inquilino
Inicio
Fin
Alquiler actual
Próxima actualización
Servicios
Gas
Luz
Agua
Municipal

La ficha debería ser un **punto central de acceso a toda la información relacionada con esa propiedad**.

8. Personas

Debe existir un módulo independiente para personas.

Una persona puede tener diferentes roles.

Por ejemplo:

- propietario;
- inquilino;
- garante;

- comprador;
- vendedor;
- contacto;
- etc.

No conviene crear una tabla/persona diferente para cada rol.

Una misma persona podría eventualmente ser:

| propietario + garante

o

| propietario + comprador.

Por eso conceptualmente conviene tener:

PERSONA

↓

roles

en lugar de:

Propietario

Inquilino

Garante

Comprador

como entidades completamente separadas.

9. Información de personas

Datos posibles:

- nombre;
- apellido;
- DNI;
- CUIT/CUIL;
- teléfono;
- WhatsApp;
- email;
- domicilio;
- observaciones;
- documentación;

- rol dentro de la inmobiliaria.

La aplicación debería permitir buscar personas rápidamente.

Por ejemplo:

"Gómez"

y encontrar todas las personas correspondientes.

10. Contratos

El módulo de contratos es otro de los núcleos del sistema.

Cada contrato debería vincular:

```
Propiedad
  ↓
Contrato
  ↓
Propietario
  ↓
Inquilino
```

y eventualmente:

```
Garantías
```

11. Datos del contrato

Un contrato debería almacenar, entre otros:

- fecha de inicio;
- fecha de finalización;
- alquiler inicial;
- alquiler actual;
- moneda;
- frecuencia de actualización;
- índice de actualización;
- fecha de próxima actualización;
- porcentaje o mecanismo especial;
- depósito;

- garantía;
- observaciones;
- estado.

Estados posibles:

- Activo
- Próximo a vencer
- Finalizado
- Rescindido
- Suspendido

12. Sistema de actualización de alquileres

Este punto es especialmente importante.

No hay que diseñarlo pensando:

"todos aumentan por ICL".

Debe existir una estructura flexible.

Por ejemplo:

```
Contrato
|
├─ Método de actualización
|   ├── ICL
|   ├── IPC
|   ├── Porcentaje fijo
|   └─ Personalizado
|
├─ Frecuencia
|   ├── 3 meses
|   ├── 4 meses
|   ├── 6 meses
|   └─ personalizada
|
└─ Próxima actualización
```

Además debería quedar registrado:

```
Alquiler anterior
Fecha
Índice utilizado
```

Esto permitiría tener un historial.

13. ICL / índices

En el proyecto original se había contemplado una sección específica para consultar el **ICL del Banco Central**.

La idea era poder tener:

```
Índices
├── ICL
├── IPC
└── otros
```

Esto permitiría que la actualización de contratos utilice los valores correspondientes.

Importante:

la lógica de índices debería estar separada de la interfaz.

Es decir, la pantalla simplemente consulta:

```
obtenerIndice("ICL", fecha)
```

y no debería tener toda la lógica del cálculo metida dentro del HTML.

14. Cobranzas

Otro módulo fundamental.

Debería permitir visualizar qué ocurrió con cada alquiler durante un determinado período.

Ejemplo:

Septiembre 2026

Propiedad	Inquilino	Alquiler	Estado
Casa A	Pérez	\$650.000	PAGADO
Duplex B	Gómez	\$720.000	PENDIENTE
Casa C	Díaz	\$580.000	PAGADO

Estados posibles:

- Pendiente
 - Pagado
 - Parcial
 - Vencido
 - En gestión
-

15. Historial de cobranzas

Cada pago debería quedar registrado.

Por ejemplo:

Contrato #154

Agosto 2026

Alquiler: \$650.000

Estado: Pagado

Fecha: 05/08/2026

Septiembre 2026

Alquiler: \$690.000

Estado: Pendiente

Esto permitiría consultar el historial completo de un contrato.

16. Deudores

Debe existir un módulo específico para deudores.

La idea es poder obtener rápidamente:

¿Quién debe plata?

Y no tener que buscar contrato por contrato.

Información:

- inquilino;
- propiedad;
- período;
- monto;
- días de atraso;
- estado;

- observaciones;
- contacto.

También podría mostrar:

TOTAL ADEUDADO

y ordenar por antigüedad.

17. Servicios

Otro módulo que se había contemplado.

Especialmente para cuando termina un contrato.

La idea es poder registrar cuestiones relacionadas con:

- luz;
- gas;
- agua;
- municipal;
- otros servicios.

También querías incorporar dentro de la propiedad campos específicos para las cuentas de:

- Gas
- Luz
- Agua
- Municipal

Esto debería estar vinculado a la propiedad y, cuando corresponda, al período contractual.

18. Servicios al finalizar contrato

Un flujo interesante sería:

Contrato finaliza

↓

Revisión de servicios

↓

Gas

Luz

Agua

Municipal

↓
¿Hay deuda?
↓
Sí / No

Esto ayudaría a evitar que una propiedad vuelva a alquilarse con cuestiones pendientes.

19. Liquidaciones a propietarios

Este era otro módulo que querías conservar.

La inmobiliaria necesita poder saber cuánto corresponde liquidarle a cada propietario.

Conceptualmente:

Alquiler cobrado
↓
- conceptos
↓
= importe a liquidar

Debería poder consultar:

- propietario;
- propiedades;
- alquiler cobrado;
- gastos;
- honorarios/comisiones;
- servicios;
- otros descuentos;
- total a liquidar.

20. Un propietario puede tener varias propiedades

Esto es importante para la arquitectura.

No hay que diseñar:

1 propietario = 1 propiedad

sino:

Propietario A

- |
- ├ Propiedad 1
- ├ Propiedad 2
- ├ Propiedad 3
- └ Propiedad 4

Por ejemplo, en algún momento se había mencionado el caso de un propietario con **13 unidades/duplexes**.

Por eso las liquidaciones deberían poder agrupar varias propiedades de un mismo propietario.

21. Impresión

La impresión es importante porque el negocio todavía utiliza documentación física.

La aplicación no debería ser exclusivamente "digital".

Debería existir una lógica de:

Ver
Editar
Guardar
Imprimir

Por ejemplo:

Recibos

Poder imprimir los recibos correspondientes al período.

La idea que habíamos manejado era **imprimir todos los recibos mensualmente**, no únicamente los contratos que tuvieron actualización.

Pero aquellos que requieran actualización deberían quedar identificados.

Por ejemplo:

✓ Recibo generado

⚠ Pendiente de actualizar

22. Estado "Pendiente de actualizar"

Este concepto puede ser muy útil.

En vez de esconder contratos que todavía no fueron actualizados, el sistema puede mostrar:

Actualización de septiembre

✓ 132 actualizados

⚠ 8 pendientes

Y permitir entrar directamente a esos contratos.

Esto evita que algo quede perdido.

23. Flujo mensual ideal

Una de las cosas que debería resolver la aplicación es el trabajo mensual de administración.

Idealmente:

INICIO DEL MES

↓

Dashboard

↓

Revisar actualizaciones

↓

Actualizar contratos

↓

Generar alquileres

↓

Registrar cobranzas

↓

Detectar deudores

↓

Generar recibos

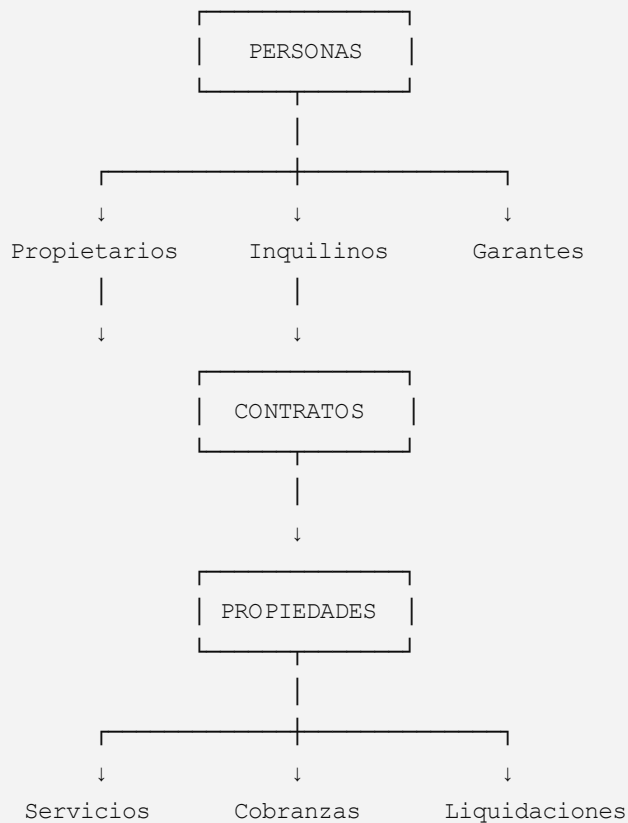
↓

Liquidar propietarios

El sistema debería ayudar a que el usuario sepa **qué tiene pendiente hacer**.

24. Relaciones entre entidades

Una arquitectura de datos razonable sería:



Pero técnicamente sería mejor que estas relaciones estén normalizadas y no duplicar información.

25. Arquitectura técnica que propondría ahora

Acá es donde cambiaría bastante respecto del primer intento.

No arrancaría nuevamente con un HTML gigante.

Propondría algo conceptualmente así:

CARUSO APP

```
|
|
| Frontend
|   | Dashboard
|   | Propiedades
|   | Personas
|   | Contratos
|   | Cobranzas
|   | Deudores
|   | Servicios
|   | Liquidaciones
|   | Configuración
|
```



```
├── Backend
│   ├── propiedades
│   ├── personas
│   ├── contratos
│   ├── cobranzas
│   ├── deudores
│   ├── servicios
│   ├── liquidaciones
│   └── indices
└── Base de datos
```

La clave es que **cada módulo tenga responsabilidades claras**.

26. Una estructura de carpetas posible

Para trabajar después con Cursor/Claude, algo así sería mucho más sano:

```
caruso-app/
├── frontend/
│   ├── components/
│   ├── pages/
│   ├── layouts/
│   ├── styles/
│   └── services/
├── backend/
│   ├── routes/
│   ├── models/
│   ├── services/
│   ├── repositories/
│   └── utils/
├── database/
│   ├── migrations/
│   └── seeds/
├── tests/
├── docs/
└── README.md
```

No digo que esta tenga que ser exactamente la estructura final. Lo importante es **separar responsabilidades**.

27. Base de datos

La base debería ser relacional.

Entidades principales:

```
persons
properties
contracts
contract_parties
rent_updates
payments
debts
services
owners
settlements
indexes
documents
```

Aunque algunos nombres pueden cambiar.

28. No duplicar información

Ejemplo de algo que hay que evitar:

```
Contrato:
Nombre inquilino = Juan Pérez

Propiedad:
Nombre inquilino = Juan Pérez
```

Porque si Juan cambia un dato, después aparecen inconsistencias.

Mejor:

```
Person
  ID 45
  Juan Pérez

Contract
  tenant_id = 45
```

La aplicación obtiene los datos desde la persona.

Esto es justamente el tipo de cosa que una buena arquitectura evita.

29. Diseño visual

La aplicación debería sentirse más como un **sistema administrativo moderno** que como una planilla Excel.

Pero tampoco como una aplicación bancaria hipercompleja.

Algo intermedio.

Sidebar

```
CARUSO
Propiedades

Dashboard
Propiedades
Personas
Contratos
Cobranzas
Deudores
Servicios
Liquidaciones

-----

Configuración
```

30. Dashboard visual

Podría tener arriba tarjetas simples:

170	8	12
Alquileres	Pendientes	Deudores

Y debajo:

ACTIVIDAD / PENDIENTES

⚠ 8 contratos necesitan actualización

⚠ 12 alquileres pendientes

✓ 142 cobranzas registradas

La pantalla debería permitir actuar, no solamente mirar estadísticas.

31. Propiedades visualmente

Una pantalla podría ser:

Propiedades

+ Nueva propiedad

[🔍 Buscar dirección, propietario...]

Filtros:

[Todos] [Alquiladas] [Disponibles] [Venta]

Casa	Paso del Rey 1234	ALQUILADA
Duplex	Moreno 450	DISPONIBLE
Local	Belgrano 850	ALQUILADA

Al hacer clic:

→ Ficha de propiedad

32. Ficha con pestañas

Una propiedad podría tener:

Información | Contrato | Cobranzas | Servicios | Documentos

Esto me parece particularmente interesante porque evita tener una pantalla kilométrica.

Por ejemplo:

Información

Datos físicos.

Contrato

Contrato actual.

Cobranzas

Historial.

Servicios

Gas/luz/agua/municipal.

Documentos

Contratos, recibos, etc.

33. Contratos

La pantalla podría tener:

Contratos

[🔍 Buscar inquilino, propiedad...]

Filtros:

[Activos] [Por vencer] [Finalizados]

Contrato	Propiedad	Inquilino	Alquiler

#00124	Duplex 4	Pérez	\$650.000
#00125	Casa 8	Gómez	\$720.000

Y un indicador:

⚠ Actualización pendiente

cuando corresponda.

34. Cobranzas

La interfaz debería ser muy operativa.

Por ejemplo:

COBRANZAS — SEPTIEMBRE 2026

[Buscar]

TOTAL

\$ XX.XXX.XXX

COBRADO

\$ XX.XXX.XXX

PENDIENTE
\$ X.XXX.XXX

Después:

Propiedad | Inquilino | Importe | Estado

Con acciones rápidas:

[Marcar pagado]
[Ver]

35. Deudores

Esta pantalla debería ser prácticamente una lista de trabajo.

DEUDORES

12 casos

Pérez Juan	
Duplex Paso del Rey 123	
2 períodos	
\$1.240.000	
[Ver contrato] [Contactar]	

36. Liquidaciones

Algo similar:

LIQUIDACIONES

Septiembre 2026

Propietario	A liquidar

Carlos Pérez	\$1.450.000
María Gómez	\$820.000
...	

Al abrir:

PROPIETARIO: Carlos Pérez

Propiedades:

- Duplex 1
- Duplex 2
- Duplex 3
- Duplex 4

Ingresos

- gastos
- honorarios

TOTAL A LIQUIDAR

37. Documentos

Aunque inicialmente no fuera el foco, la arquitectura debería dejar preparado el terreno para documentos.

Por ejemplo:

Contrato

└─ PDF

Recibo

└─ PDF

Liquidación

└─ PDF

No hace falta construir un gestor documental enorme desde el principio.

Pero sí conviene **no diseñar la base de datos de manera que después sea imposible agregarlo**.

38. Lo que NO debería hacerse

Hay varias lecciones del proyecto anterior.

No hacer:

✗ Un único HTML gigante.

✗ Meter toda la lógica dentro de las páginas.

- ❌ Duplicar datos.
- ❌ Cambiar la arquitectura completa mientras se agregan funcionalidades.
- ❌ Refactorizar 10.000 líneas de golpe.
- ❌ Modificar una funcionalidad existente sin comprobar que las demás sigan funcionando.
- ❌ Crear una interfaz llena de botones porque "algún día pueden servir".
- ❌ Diseñar pensando primero en el programador.

39. Estrategia de desarrollo

Yo lo haría por etapas.

Etapa 1 — Diseño visual

Antes de base de datos real:

```
Dashboard
Propiedades
Ficha propiedad
Personas
Contratos
Cobranzas
Deudores
Liquidaciones
```

Con **datos ficticios**.

Objetivo:

Ver cómo se siente la aplicación.

Esta es justamente la etapa que vos querés hacer ahora con Claude.

Etapa 2 — Arquitectura

Definir:

- entidades;
- relaciones;
- API;
- estructura de carpetas;

- autenticación;
 - permisos;
 - base de datos.
-

Etapa 3 — Propiedades + personas

Construir primero:

Propiedades
+
Personas

Porque prácticamente todo lo demás depende de ellas.

Etapa 4 — Contratos

Después:

Propiedad
+
Propietario
+
Inquilino
=
Contrato

Etapa 5 — Actualizaciones

Agregar:

ICL
IPC
frecuencia
historial

Etapa 6 — Cobranzas

Después:

Contrato

↓

Período

↓

Importe

↓

Pago

Etapa 7 — Deudores

Los deudores deberían salir de la información de cobranzas.

No crear un sistema paralelo de deudores.

Por ejemplo:

Cobranzas

↓

Pago vencido

↓

DEUDOR

Etapa 8 — Liquidaciones

Usar cobranzas reales para calcular:

Cobrado

-

descuentos

-

gastos

=

liquidación

40. Algo muy importante: primero prototipo visual

Para lo que querés hacer ahora, **yo no le pediría a Claude que construya inmediatamente toda la aplicación.**

Le pediría primero:

"Construime un prototipo visual navegable con datos ficticios."

Que tenga:

- sidebar;
- dashboard;
- propiedades;
- ficha de propiedad;
- contratos;
- cobranzas;
- deudores;
- liquidaciones;
- personas.

Y que todo sea navegable.

Por ejemplo:

```
Dashboard
  ↓
Propiedades
  ↓
Duplex 123
  ↓
Contrato
  ↓
Inquilino
  ↓
Cobranzas
```

Aunque todavía no haya base de datos.

Eso te permite **mirar la aplicación como usuario** antes de decidir la arquitectura definitiva.

41. Cómo usaría Claude + Cursor

Acá creo que podemos mejorar bastante respecto del año pasado.

Claude

Lo usaría para:

- pensar arquitectura;
- revisar estructura;
- diseñar componentes;

- detectar problemas;
- generar prototipos;
- revisar código;
- discutir decisiones técnicas.

Cursor

Lo usaría para:

- implementar;
- modificar archivos;
- conectar módulos;
- refactorizar partes puntuales;
- trabajar directamente sobre el proyecto.

Y la regla sería:

Claude piensa/revisa. Cursor implementa.

No necesariamente siempre, pero como división general me parece muy buena para tu caso.

42. El "contrato" que debería tener el proyecto

También haría algo que antes no teníamos tan estructurado:

Un archivo:

```
PROJECT_RULES.md
```

que diga cosas como:

```
# Reglas del proyecto Caruso

1. No eliminar funcionalidades existentes sin autorización.
2. No modificar módulos no relacionados con la tarea.
3. Mantener la búsqueda global de propiedades.
4. Mantener compatibilidad con impresión.
5. No duplicar entidades.
6. Toda modificación debe preservar funcionalidades existentes.
7. La interfaz debe priorizar PC.
8. No agregar complejidad innecesaria.
9. Separar UI, lógica y acceso a datos.
10. Antes de refactorizar, analizar dependencias.
```

Esto puede parecer una boludez, pero para trabajar con agentes de IA es **muy importante**.

43. Cómo debería sentirse la aplicación

Si tuviera que resumir el concepto visual en una frase:

"Una mezcla entre un CRM sencillo y un sistema administrativo de inmobiliaria, pero adaptado específicamente a cómo trabaja Caruso."

No:

SAP.

No:

Excel con botones.

No:

una app futurista llena de gráficos.

Algo limpio y profesional.

44. Prioridades reales

Yo pondría las prioridades así:

Nivel 1 — Imprescindible

- Propiedades
- Personas
- Contratos
- Búsqueda
- Cobranzas
- Deudores
- Actualizaciones
- Impresión

Nivel 2 — Muy importante

- Liquidaciones
- Servicios
- Historial

- Alertas
- Dashboard

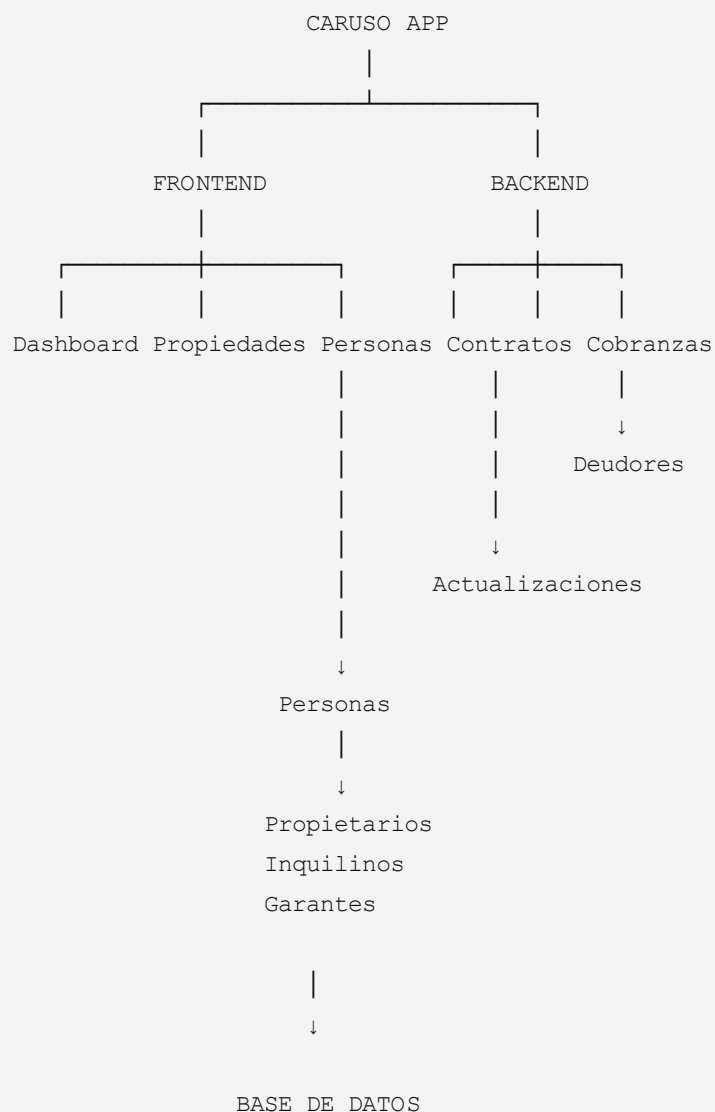
Nivel 3 — Futuro

- documentos;
- automatizaciones;
- WhatsApp;
- CRM;
- integración con portales;
- IA;
- reportes avanzados.

Esto es importante porque **no conviene intentar construir todo junto**.

45. Arquitectura conceptual final

En una visión general:



Y todo conectado, pero sin mezclar responsabilidades.

46. Lo que yo haría AHORA

No empezaría todavía con Cursor.

Haría esta primera etapa:

Claude → prototipo visual navegable → lo mirás → corregimos UX → recién después arquitectura definitiva → Cursor.

Porque hay una ventaja enorme:

Si después descubrí:

"Che, esta ficha de propiedad debería mostrar primero el contrato."

lo cambiás cuando todavía son datos ficticios.

No después de haber construido:

- base de datos;
- APIs;
- autenticación;
- lógica de contratos;
- cobranzas;
- etc.

Prompt maestro para Claude

Te dejo incluso un prompt inicial que podés pegarle a Claude:

Prompt maestro para Claude — Caruso Propiedades



Quiero desarrollar una aplicación web interna para una inmobiliaria llamada Caruso Propiedades, ubicada en Paso del Rey, Moreno, Argentina.

Quiero que actúes inicialmente como arquitecto de software + diseñador UX/UI.

IMPORTANTE: en esta primera etapa NO quiero que construyas todavía la aplicación completa ni una base de datos real. Quiero comenzar con un PROTOTIPO VISUAL NAVEGABLE utilizando datos ficticios, para poder evaluar cómo se ve y cómo se siente la aplicación antes de definir la implementación definitiva.

CONTEXTO

Caruso Propiedades administra aproximadamente 170 alquileres, además de propiedades en venta y otras operaciones inmobiliarias.

La aplicación debe centralizar la gestión de:

- Dashboard
- Propiedades
- Personas
- Contratos
- Cobranzas
- Deudores
- Actualizaciones de alquileres
- Servicios
- Liquidaciones a propietarios
- Impresión de recibos y documentación

La aplicación será utilizada principalmente desde PC por personal de la inmobiliaria. La prioridad es que sea rápida, clara, práctica y fácil de aprender.

NO quiero un sistema excesivamente complejo ni una interfaz llena de estadísticas innecesarias.

MÓDULO PROPIEDADES

Debe permitir:

- crear propiedad;
- editar propiedad;
- eliminar propiedad;
- buscar propiedad;
- visualizar ficha completa;
- consultar propietario;
- consultar contrato;
- consultar inquilino;

- consultar cobranzas;
- consultar servicios.

La búsqueda de propiedades es MUY IMPORTANTE.

Debe existir una búsqueda rápida tipo Google que permita buscar por dirección, localidad, propietario, inquilino, tipo de propiedad o ID.

MÓDULO PERSONAS

Debe existir una entidad general de Personas.

Una persona puede tener distintos roles:

- propietario;
- inquilino;
- garante;
- comprador;
- vendedor;
- contacto.

No quiero duplicar personas según el rol.

MÓDULO CONTRATOS

Un contrato relaciona:

- propiedad;
- propietario;
- inquilino;
- garantías;
- fechas;
- alquiler;
- condiciones de actualización.

Debe contemplar diferentes mecanismos de actualización:

- ICL;
- IPC;
- porcentaje fijo;
- otros mecanismos personalizados.

También debe contemplar distintas frecuencias:

- 3 meses;
- 4 meses;
- 6 meses;
- personalizada.

Debe existir historial de actualizaciones.

MÓDULO COBRANZAS

Debe permitir registrar y consultar:

- período;
- importe;
- estado;
- fecha de pago;
- pagos parciales;
- vencimientos.

Estados posibles:

- pendiente;
- pagado;
- parcial;
- vencido;
- en gestión.

MÓDULO DEUDORES

Debe mostrar automáticamente los casos donde existen pagos vencidos o pendientes.

No quiero mantener una base de deudores independiente y duplicada. Los deudores deben surgir de la información de cobranzas.

MÓDULO SERVICIOS

Debe permitir registrar información relacionada con:

- gas;
- luz;
- agua;
- municipal;
- otros.

También debe contemplar la revisión de servicios al finalizar un contrato.

MÓDULO LIQUIDACIONES

Un propietario puede tener múltiples propiedades.

El sistema debe permitir agrupar sus propiedades y calcular/visualizar liquidaciones.

Debe contemplar:

- alquiler cobrado;
- gastos;
- descuentos;
- honorarios/comisiones;
- otros conceptos;
- total a liquidar.

IMPRESIÓN

La aplicación debe estar pensada para poder imprimir recibos y documentación.

Los recibos deben poder generarse para todos los alquileres del período.

Los contratos cuya actualización todavía no se haya procesado deben quedar claramente identificados como "Pendiente de actualizar".

DISEÑO

Quiero una interfaz moderna, limpia y profesional.

No quiero que parezca Excel.

Tampoco quiero una interfaz exageradamente tecnológica.

Debe sentirse como un sistema administrativo/CRM sencillo diseñado específicamente para una inmobiliaria.

Propongo inicialmente:

- sidebar lateral;
- dashboard;

- buscador global;
- tablas limpias;
- fichas detalladas;
- pestañas cuando una entidad tenga mucha información;
- botones de acción claros;
- estados mediante badges;
- diseño responsive, pero priorizando PC.

PROPUESTA DE NAVEGACIÓN

Dashboard

Propiedades

Personas

Contratos

Cobranzas

Deudores

Servicios

Liquidaciones

Configuración

REGLAS IMPORTANTES DEL PROYECTO

1. No crear un HTML gigante.
2. Separar interfaz, lógica de negocio y acceso a datos.
3. No duplicar información innecesariamente.
4. No eliminar funcionalidades existentes durante futuros cambios.
5. La búsqueda de propiedades es una funcionalidad crítica y debe mantenerse.
6. La impresión es una funcionalidad importante.
7. No agregar complejidad innecesaria.
8. Priorizar estabilidad sobre sofisticación.
9. Diseñar primero la experiencia de usuario y después la arquitectura definitiva.
10. No construir toda la aplicación de una vez.

PRIMER OBJETIVO

Construí un prototipo visual navegable con datos ficticios que incluya como mínimo:

1. Dashboard
2. Listado de propiedades
3. Buscador de propiedades
4. Ficha de propiedad
5. Personas
6. Contratos
7. Cobranzas
8. Deudores
9. Liquidaciones

Quiero poder navegar entre estas pantallas y visualizar cómo funcionaría el sistema.

Antes de escribir una arquitectura definitiva, analizá esta especificación, detectá inconsistencias o entidades faltantes y proponé una arquitectura de alto nivel.

No implementes todavía autenticación, base de datos real, integraciones externas ni automatizaciones.

Primero quiero VER la aplicación.

Y una recomendación clave

Yo **no le daría a Claude todo esto y le diría "hacé la app"**.

Le daría el prompt y le pediría primero el prototipo.

Después lo mirás y me podés traer acá capturas o directamente el código/proyecto y hacemos una segunda pasada diciendo cosas concretas como:

"Esta pantalla me gusta, pero la ficha de propiedad quiero que se parezca más a esto."

"Cobranzas debería ser mucho más rápida."

"El dashboard me sobra."

"Quiero que desde una propiedad pueda llegar al inquilino en un clic."

Ahí podemos ir **diseñando la experiencia real de Caruso**, antes de meternos nuevamente en el quilombo técnico de implementar todo.

Y esta vez, la arquitectura la hacemos **alrededor de la aplicación que querés usar**, no al revés.